

Informatik BW: Python Crash Course C

VU Informatik BW, WS24

Thorsten Ruprechter

- feedbackr.io (Variablen, Datentypen, Operatoren)
- Lists
- for-Loops (Basics)

Kurze Wiederholung: if-Statements, Lists, for-Loops



fbr.io/infobw3

Zeit: 8 Minuten



- Loops (ff.)
- Datenstrukturen (ff.)
 - Sets
 - Dictionaries
- Functions (Basics)
- Assignment 1

for-Loops (0): Basics von for-Schleifen

- Iterieren/Durchlaufen von Sequenzen (z. B. Listen)
- **FÜR** jedes Element **IN** Sequenz:
Führe Anweisungen aus
- **Python-Syntax für for-Loops (Schleifen):**

```
for item in list_of_items:  
    print('This will be repeated.')
```
- Laufvariablen können beliebigen Namen haben
- Wiederholter Code-Block mit beliebig vielen Anweisungen

for-Loops (1): Beispiel

```
lectures = ['Maths', 'English', 'Computer Science']  
  
for lecture in lectures:  
    print(f'I love {lecture}')
```

for-Loops (2): Beispiel

```
lectures = ['Maths', 'English', 'Computer Science']
```

```
for lecture in lectures:
```

```
    print(f'I love {lecture}')
```

Legende

Iterierte Sequenz (z. B. Liste)

Definition der Sequenz

for-Loop Definition (Syntax)

Wiederholter Code-Block

Lauf-/Schleifen-Variable

for-Loops (3): Zählen mit `range()`

```
print('Count from one to ten!')  
for number in range(1,11):  
    print(number)
```

for-Loops (4):

Index mit Elementen ausgeben mit `enumerate`

```
lectures = ['Maths', 'English', 'Computer Science']  
  
for index, lecture in enumerate(lectures):  
    print(f'Lecture {index}: {lecture}')
```

for-Loops (5): Iterieren über Strings („Buchstabieren“)

```
string = 'Informatik BW'
```

```
for char in string:  
    print(number)
```

for-Loops (6):

Mehrere Listen auf einmal durchlaufen mit `zip`

```
lectures = ['Maths', 'English', 'Computer Science']  
teachers = ['Browne', 'Orwell', 'Turing']
```

```
for lecture, teacher in zip(lectures, teachers):  
    print(f'Lecture {lecture}held by {teacher}')
```

Loops: Keywords zum Schleifen- und Programmablauf

- `pass`
 - „leerer“ Command, tut nichts
- `break`
 - Schleifenabbruch
 - Sofortiges Beenden der „innersten Schleife“
- `continue`
 - Sprung zu nächster Iteration
 - Abbruch aktueller Durchlauf „innerster Schleife“

Loops: Beispiele

Siehe Code-Beispiele

- Ablegen von mehreren Werten
- Für Python-Grundlagen:
 - Lists (siehe „Python Crash Course B“)
 - Tuples (siehe „Python Crash Course B“)
 - Sets
 - Dictionaries

Sets/Mengen (0): Basics

- **Ungeordnete Sammlung eindeutiger Elemente**
 - Mathematische Menge
 - Keine Doppelgänger
- Definition durch `{}` oder `set()`
 - Leeres set: `empty = set()`
- Verwendung
 - Schnelle Überprüfung ob Elemente enthalten, Doppelgänger löschen, Mengenlehre, ...
 - Kein „direkter“ Elementzugriff (z. B. per Index)
- Anzahl Elemente: `len()`

```
colors      = {'blue', 'red'}  
characters = set('Hello, World')
```

Sets (1): Hinzufügen und Löschen

■ Hinzufügen

- Einzel: `colors.add('red')`
- Mehrere: `colors.update(['orange', 'black'])`

■ Löschen

- Mit Überprüfung
 - `colors.remove('blue')`
- Ohne Überprüfung
 - `colors.discard('blue')`
- Alle löschen
 - `colors.clear()`

Sets (2): Mengenoperationen (Auswahl)

- Schnittmenge (AND)

- `x.intersection(y)` oder `x & y` $\Rightarrow$ `{'b'}`

- Vereinigungsmenge (OR)

- `x.union(y)` oder `x | y` $\Rightarrow$ `{'a', 'b', 'd'}`

- Membership/Zugehörigkeit

- `'b' in y` $\Rightarrow$ `True`

- Differenz

- `x - y` $\Rightarrow$ `{'a'}`

- Mehr Operatoren: [Python-Doc](#)

```
x = {'a', 'b'}
y = {'b', 'd'}
```

Dictionaries/“Wörterbücher“ (0)

- Sammlung von ungeordneten Schlüssel-Wert Paaren (Key-Value pairs)
- Wichtiges Konzept in Python
- Zugriff über eindeutige Keys
 - Beliebiger Datentyp: Integer, String, etc.
- Pro Key **EIN** Value:
 - Integer, String, List, Set, Dictionary, etc.
- `KeyError` wenn Element bei Zugriff (per `[]`) nicht vorhanden
 - Zugehörigkeitsprüfung durch `in` und `not in`



Dictionaries (1): Real-World Beispiele



https://www.deutschlandfunkkultur.de/zentrum-fuer-digitale-lexikographie-die-gesamte-deutsche.1008.de.html?dram:article_id=439553



<https://pxhere.com/en/photo/529235>

Dictionaries (2): Definition, Zugriff, Zuweisung, Löschen

■ Definition

```
students = {123: 'Maria', 131: 'Tim', 42: 'Max'}
```



■ Zugriff

- `students[123]`
- `students.get(123)`

■ Einfügen/Ändern

- `students[20] = 'Lisa'`
- `students[123] = 'Moritz'`

■ Löschen (per Key)

- `del students[123]`

Dictionaries (3): Beispiel

```
students = {123: 'Maria', 131: 'Tim', 42: 'Max'}  
# A  
students[668] = 'Martina'  
# B  
del students[123]  
# C  
students[131] = 'Anna'  
# D  
del students[9999]  
# E  
print(f'e: {students[42]}')
```

Dictionaries (4): Erhalten und Iterieren aller Einträge

■ Retournieren von:

- Keys

```
- students.keys()
```

- Values

```
- students.values()
```

- Key-Value Tuples

```
- students.items()
```

■ Durchlaufen aller Schlüssel und Werte (ungeordnet):

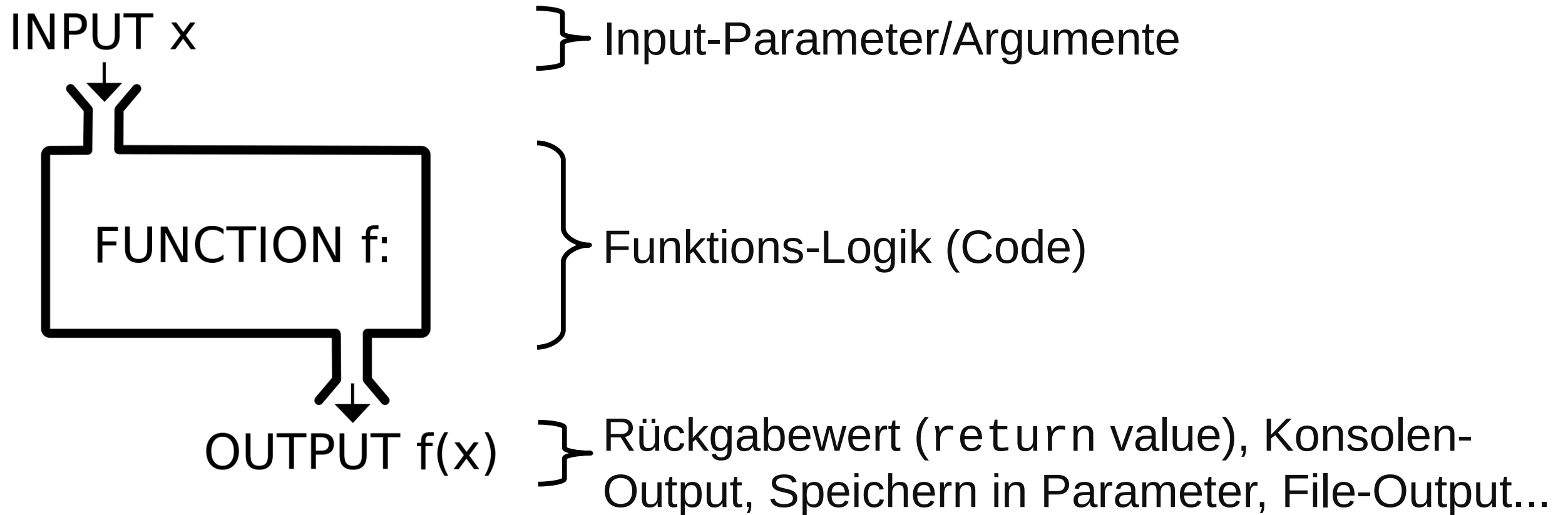
```
for key, value in students.items():  
    print(f'{key}: {value}')
```

```
students = {123: 'Maria',  
            131: 'Tim',  
            42: 'Max'}
```

Sets & Dictionaries: Beispiele

Siehe Code-Beispiele

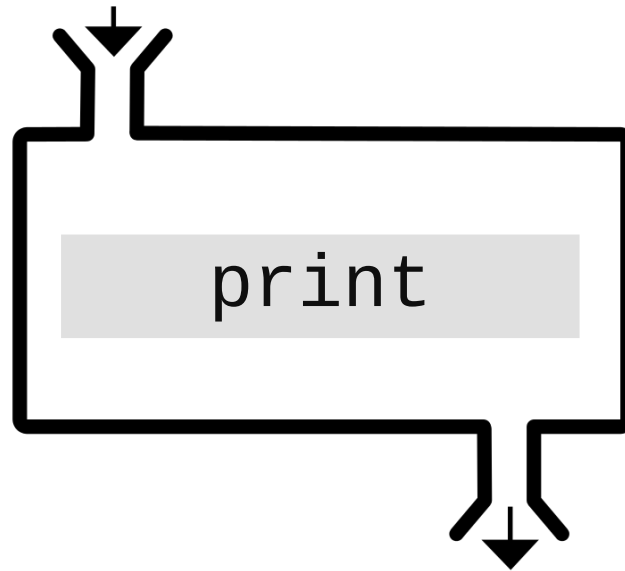
Functions (0): Grundkonzept



Functions (1):

Beispiel `print('This', 'is', 'a', 'Test')`

Argumente: `'This', 'is', 'a', 'Test'`

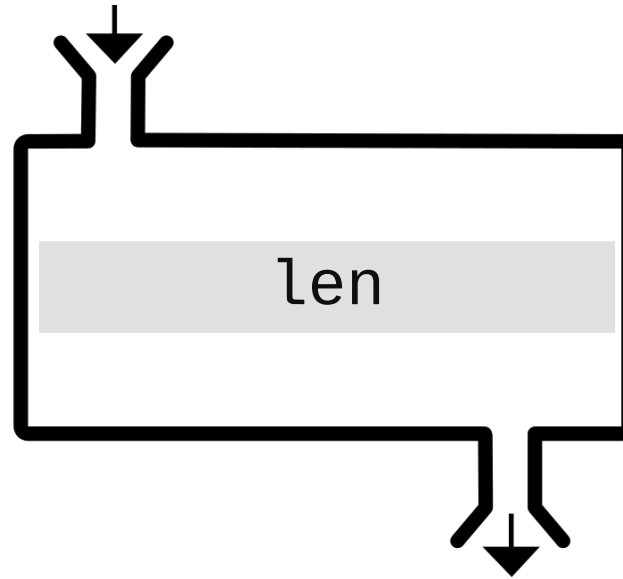


In Konsole: `This is a Test`

Functions (2):

Beispiel `len([4, 8, 'This', 'Test'])`

Argumente: `[4, 8, 'This', 'Test']`



Rückgabewert
(return-value)

4

Funktionen/Functions (3)

- „Benannter Codeblock“ mit spezifischer Aufgabe
 - Wiederverwendbar, einfacher lesbar, besser wartbar
 - Beispiele von Python-Funktionen: `len()`, `print()`, `input()`, ...
- Definiert durch Schlüsselwort **def**, Name der Funktion, Parameter-Liste
 - Code-Block in Funktion: eingerückt!
- Ausführung bei Aufruf (call), danach Programm fortgesetzt
- Rückgabe von Werten an Hauptprogramm möglich (`return`)

```
def test_function(param):  
    print(f'You passed "{param}" to this function')  
  
test_function(13)
```

Functions (4):

Einfaches Code-Beispiel für Definition und Aufruf

```
def test_simple():  
    print(f'Hi everyone! ' )  
  
def test_param(param):  
    print(f'You passed "{param}" to this function')  
  
test_simple()  
test_param('Hello! ')
```

Functions (5): Funktionen vs Methoden

- Funktion
 - „Eigenständige“ Funktionalität
 - `print()`, `len()`, `range()`, ...
- Methode
 - Gehört zu Objekt/Variable
 - Auf Variable aufgerufen („Method call“)
 - Hat Zugriff auf Daten/Werte in Variable
 - `string.upper()`, `item_list.append()`,
`dictionary.keys()`, ...

Functions (6a): Rückgabewerte (return values)

- „Berechnungen“ in Funktionen, Rückgabe an Hauptprogramm
- Rückgabe von Werten aus Funktionen
 - Keyword: `return`
 - Sofortige Rückkehr zum Hauptprogramm (auch ohne Wert möglich)

```
def format_name(first, last):  
    full_name = f'{first} {last}'  
    full_name = full_name.title()  
    return full_name
```

```
name = format_name('aRn0lD', 'sChWaRzEnEgGeR')
```

Functions (6b): Rückgabewerte (return values)

- „Berechnungen“ in Funktionen, Rückgabe an Hauptprogramm
- Rückgabe von Werten aus Funktionen
 - Keyword: `return`
 - Sofortige Rückkehr zum Hauptprogramm (auch ohne Wert möglich)

```
def format_name( 'aRnOlD', ' sChWaRzEnEgGeR' )  
    full_name = 'aRnOlD sChWaRzEnEgGeR'  
    full_name = 'Arnold Schwarzenegger'  
    return 'Arnold Schwarzenegger'
```

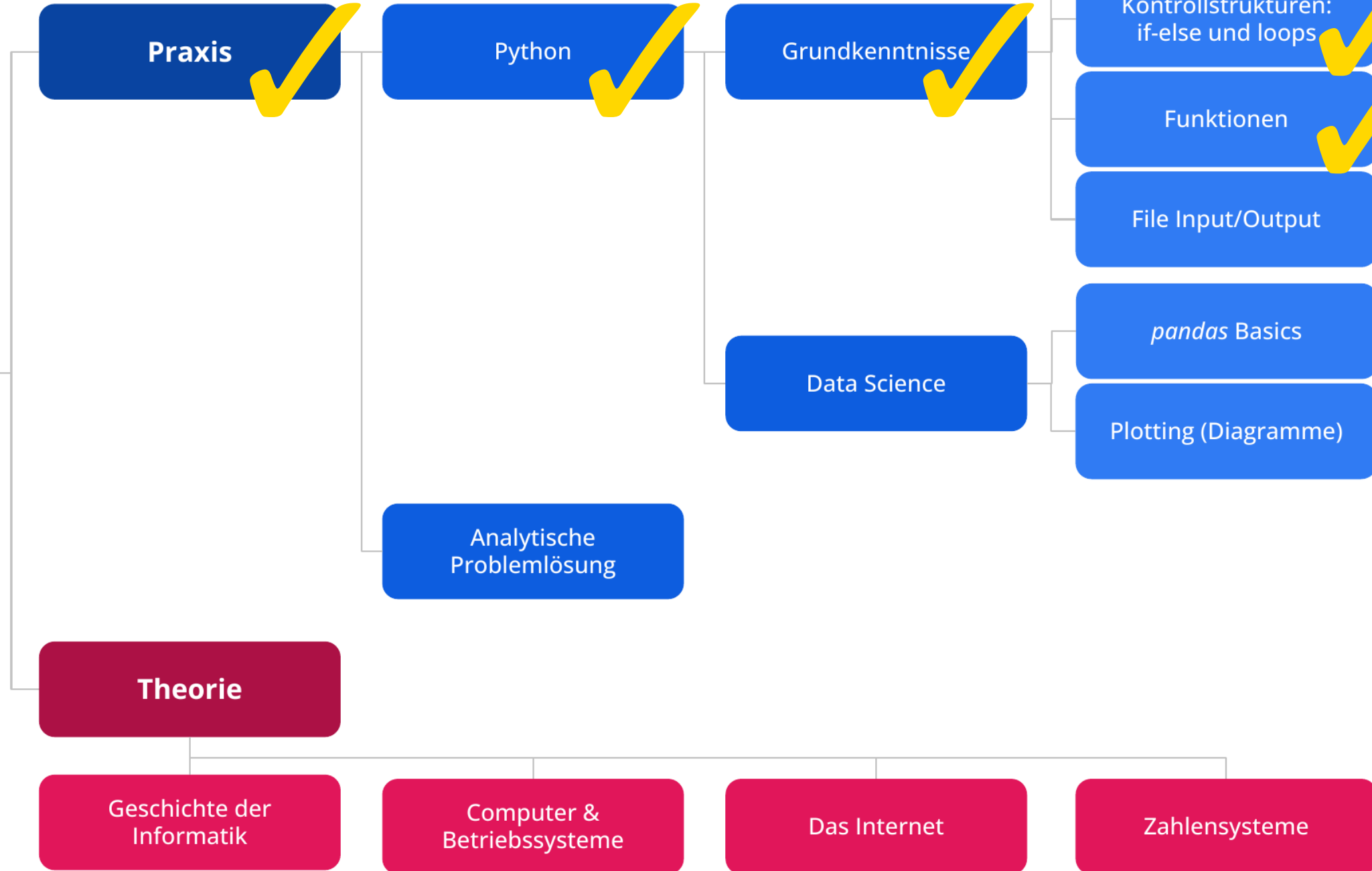
```
name = 'Arnold Schwarzenegger'
```

Functions (7): Mehr zu `return` values

- „Kompliziertere“ Rückgabewerte möglich
 - Listen, Tuples, Dictionaries, etc.
- Rückgabe von mehreren Variablen gleichzeitig möglich
 - `return x, y, z`

Functions – Parameters/Arguments, Wartbarkeit, return-Values: Erste Beispiele

Siehe Code-Beispiele



Assignment 1:

“Grundlegende Programmierkonzepte“

(24 Punkte)

Assignment 1: Inhalte

- Üben der bisherigen VO-Inhalte
 - Variablen, Datentypen, Funktionen, if-else, Loops, Listen, Strings, Sets, Dictionaries ...
 - “Code-Snippets” als separate Aufgaben
 - Basics zu Funktionen und Struktur: 6 Punkte (1P + 2P + 3P)
 - Lists und Schleifen: 8 Punkte (3P + 5P)
 - Datenstrukturen und Strings: 10 Punkte (2P + 4P + 4P)
- => 24 Punkte**

Assignment 1: Abgabe

- Exakte Beschreibung in `assignment1_angabe.html`
- Abgabe von **EINEM** “Jupyter Notebook”-File: `assignment1.ipynb`
 - Deadline: 07.11.2024 (23:59)
- Hinweis (siehe Angabe):
 - Anforderungen und Output sind EXAKT wiederzugeben
 - Kommentar-Header muss in Datei enthalten sein
 - Coding-Standard muss eingehalten werden

Assignment 1: Automatisierte Tests

- Automatisiertes Testing für **Teil der Funktionalität**
 - **Alle Tests erfolgreich $\neq$ Alle Punkte!**
 - Für finale Abgabe: Mehr Testcases
- Test-Runs jeden Dienstag- und Freitagabend (ab 22.10.)
 - Grundlage für Tests: Abgaben vor Dienstag/Freitag 18:00
- Testreport per automatischer Mail bis nächsten Vormittag (~9:00)
 - Fragen zu Testreport: Tutorium!
 - Wie liest man den Testreport?
 - <https://hackmd.io/@info-tug/Skd-6JUHF>

Informatik BW: Python Crash Course C

VU Informatik BW, WS24

Thorsten Ruprechter